HARNESS CAPABILITY NOTE

🧠 Harness is a control plane, not the brain ⚙️ (Executive Conclusion)

- 🚀 The API/model provides coding ability, inference, and research horsepower.
- 🧭 Your RDF triples provide authority: approved taste, rules, beliefs, patterns, and "do/don't treat this as true" boundaries.
- 🔗 Harness combines them by retrieving accepted graph authority first, adding supporting memory second, then sending a constrained packet to Codex/Grok/Claude/Hermes.
- ✅ So RDF does not make the model smart. It makes the model answer through your approved judgment layer.

⚠️ What the app can already do (Consequence)

Current macOS Harness has:

- 🧠 Multi-backend execution: Codex, Grok, Claude API, local Hermes/Ollama.
- 📒 Run ledger: prompt, answer, backend, authority hits, memory hits, trace events, evals, candidates.
- 🕸️ Authority retrieval: SPARQL `/accepted` named graph when available, bundled TTL fallback otherwise.
- 🧱 Memory separation: accepted graph authority is labeled separately from supporting notes.
- 📦 Candidate review: pending claims can be approved/rejected, validated as Turtle/SHACL, appended to accepted graph, and best-effort posted to Fuseki.

The key implementation is **HarnessRunService.swift**: authority retrieval, supporting memory, model execution, evaluation, candidate extraction, ledger save.

The missing bridge is "RDF as policy," not just "RDF as content" (Recommendation)

The next step is to make accepted triples drive agent behavior more directly: tool permissions, coding style, research depth, delegation defaults, output evaluations, and repair loops. Right now Harness mostly uses RDF as prompt authority plus traceable context. To become "Codex with Adam judgment," the graph needs to become the policy/eval layer around coding and research actions, not only a memory packet sent before a chat response.

Supporting Evidence on Request (Supporting Evidence on Request)

Relevant files:

- **AuthorityRetrieval.swift**: tries live SPARQL, then falls back to bundled TTL.
- **AuthorityRetrieval.swift**: builds the model packet and labels accepted authority separately from supporting memory.
- **ReviewQueueStore.swift**: approval path from queue claim to validated Turtle to accepted graph.
- **AgentRunner.swift**: routes constrained packets to Codex, Grok, Claude, or local Hermes.

Important caveat: **reconciliation-report.md** says the bulk Fuseki graph should not be treated as accepted Adam truth. The current code expects a named `/accepted` graph; the audit says the large Fuseki graph was loaded into the default graph with no named graphs.

Verification: `swift test --package-path Packages/OntologyKit` passed 32 tests, `xcodebuild -project Harness.xcodeproj -scheme Harness -destination 'platform=macOS' build` succeeded, and the worktree stayed clean.

```
Rule: conn-016
Adam Pattern Step: 1
```

Generated from the Harness capability summary.